

WHITE STORK MIGRATION VISUALISER

Complete Application Guide

A full walkthrough of what the app does, how it's built, why every technology was chosen, and how the code works. Written for everyone — technical or not.

Study: White Stork Portugal Adults 2022 • *Ciconia ciconia* • 3 Birds Tracked

1. What Is This Application?

The White Stork Migration Visualiser is a web application that tracks the real-life journeys of wild storks across the sky — showing where they flew, how fast, where they stopped, and what their GPS tags recorded.

Think of it like Google Maps, but instead of tracking your own journey, it tracks the migration of white storks fitted with GPS tracking devices in Portugal in 2022. Scientists attach tiny GPS tags to birds. Every few hours, the tag records the bird's exact location, speed, altitude, and battery level — and sends it back via mobile network.

■ *In plain terms: A bird has a GPS tracker. The tracker sends data. This app collects that data, draws the bird's path on a map, and shows you stats about its journey — all accessible from a browser on any device.*

2. The Three Tracked Storks

Three individual White Storks (*Ciconia ciconia*) were tracked in Portugal during May 2022, as part of the 'White Stork Portugal Adults 2022' study.

Bird Name	GPS Fixes	Distance	Max Speed	Max Altitude	Period
Castro_O284	30	0.09 km	1.0 km/h	173 m	8 May 2022
Mineiro_O283	30	2.72 km	45.0 km/h	605 m	7–8 May 2022
Perneta_O285	30	1.10 km	18.0 km/h	184 m	9 May 2022

3. The Tech Stack — Why Each Tool Was Chosen

Python [Language]

What it is: Python is a programming language — the set of instructions the computer reads. Think of it as the language this entire application is written in, like English is the language of this document.

Why we used it: Python has the best scientific data tools — pandas for spreadsheet data, geopandas for map data, scipy for statistics. For a project that processes GPS coordinates and calculates migration routes, Python was the natural choice.

Flask [Web Framework]

What it is: Flask is a tool that turns a Python program into a website. Without Flask, Python can't respond to web browsers. Flask listens for requests (someone opens a page) and sends back the right content.

Why we used it: Flask is lightweight and gives full control — perfect for a focused academic project. It handles user login, page routing (which URL shows which page), and the API endpoints that feed data to the map.

MySQL [Database]

What it is: MySQL is a database — a structured place to store information that can be searched and retrieved fast. Think of it as a very organised filing cabinet that the application opens whenever it needs to look something up.

Why we used it: The app needs to store user accounts (with secure hashed passwords) and track uploaded files. MySQL is mature and reliable. It pairs naturally with Flask via SQLAlchemy, which lets Python talk to the database without writing raw SQL queries.

Pandas & GeoPandas [Data Processing]

What it is: Pandas is like Excel inside Python — it reads spreadsheets, filters rows, calculates averages. GeoPandas is the same thing but for geographic data — it understands coordinates and can measure real distances on the Earth's surface.

Why we used it: The raw stork data arrives as an Excel file. Pandas reads it and cleans it. GeoPandas works out actual distances between GPS points on the curved surface of the Earth (not just straight-line arithmetic).

GeoJSON [Map Format]

What it is: GeoJSON is a file format designed to store geographic shapes — points, lines, and areas — in a way that mapping tools can read. It's just a text file with coordinates formatted in a special way.

Why we used it: The migration tracks need to be drawn on a map. GeoJSON is the standard format — all modern map libraries understand it natively. The app converts GPS data into GeoJSON so the frontend map can draw the routes.

HTML / CSS / JavaScript [Frontend]

What it is: These three languages build what you see in the browser. HTML is the structure (skeleton), CSS is the styling (colours, layout), and JavaScript is the behaviour (what happens when you click a button, how the map responds).

Why we used it: Every website uses these — there's no alternative for browser-based interfaces. The app's JavaScript fetches data from the Flask API and renders it on the map and charts without reloading the page.

4. Architecture — How All Pieces Connect

The application follows a classic three-layer architecture — a browser at the front, a Python server in the middle, and a database at the back.

Layer	Component	Role
Frontend	Browser + JavaScript	What the user sees and interacts with
Server	Flask (Python)	Handles requests, runs business logic
Data	MySQL Database	Stores users and file records
Files	GeoJSON / Excel	Stores pre-processed map data

5. Data Flow — GPS Tag to Map

Step 1 — GPS tag records a fix

Every few hours the tag on the stork's back locks onto satellites and records: latitude, longitude, altitude, speed, heading, battery level, and temperature. This happens automatically — the bird doesn't notice.

Step 2 — Data arrives as an Excel file

The tag transmits data via mobile network. Scientists export it as `data.xlsx` — a spreadsheet with 99 rows and 30 columns. This file lives in the `data/` folder.

Step 3 — `preprocess.py` transforms it

When the app starts, `preprocess.py` reads the Excel file, groups fixes by bird, calculates distances and stopovers using GeoPandas, then writes `tracks.geojson`, `stopovers.geojson`, `stats.json`, and `timeline.json`. This heavy calculation only runs once.

Step 4 — Flask serves the data via API

When the browser opens the map page, JavaScript calls API endpoints like `/api/tracks`. Flask reads the GeoJSON files and returns them as JSON responses — like a waiter bringing food from the kitchen to your table.

Step 5 — JavaScript draws it on the map

The browser's JavaScript receives the GeoJSON and uses a map library to draw migration routes as lines, place markers at stopovers, and render charts. The user can click, zoom, and explore.

6. Key Code Explained

How a page route works (app.py)

A 'route' links a URL to a Python function. When someone visits /dashboard, Flask runs this function.

```
@app.route('/dashboard')
@login_required
def dashboard():
    files = UploadedFile.query.filter_by(
        uploaded_by=current_user.id
    ).all()
    return render_template('dashboard.html', files=files)
```

■ *@app.route('/dashboard')* means: when someone visits /dashboard, run this function. *@login_required* means: only if they're logged in — otherwise redirect to login. The function fetches files from the database and sends the dashboard HTML with those files included.

How the database model works (models.py)

A 'model' is a Python class describing a database table. SQLAlchemy creates the table automatically.

```
class User(db.Model, UserMixin):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(80), unique=True)
    password = db.Column(db.String(255), nullable=False)
    role = db.Column(db.String(20), default='researcher')

    def set_password(self, raw_password):
        self.password = bcrypt.generate_password_hash(raw_password)

    def check_password(self, raw_password):
        return bcrypt.check_password_hash(self.password, raw_password)
```

■ *Passwords are NEVER stored as plain text — bcrypt scrambles them into a hash. Even if someone stole the database, they couldn't read the passwords. check_password verifies a login by re-scrambling the typed password and comparing the hashes.*

How GPS data becomes map-ready (preprocess.py)

Raw spreadsheet data is transformed into GeoJSON format that the map understands.

```
gdf = gpd.GeoDataFrame(df,
    geometry=gpd.points_from_xy(
```

```

        df['location-long'],
        df['location-lat']
    )
)
for bird_id, group in gdf.groupby('individual-local-identifier'):
    track = group.sort_values('timestamp')
    line = LineString(track.geometry.tolist())
    tracks_features.append({
        'type': 'Feature',
        'properties': {'bird_id': bird_id},
        'geometry': mapping(line)
    })

```

■ We tell GeoPandas that location-long and location-lat columns are GPS coordinates. We group all rows by bird name, sort them by time (so the line goes in the right direction), connect the points into a line, and save it as GeoJSON — exactly the format a map library expects.

7. Application Features

Migration Map: Interactive map showing all three birds' GPS tracks, stopover locations, and individual data points. Click any marker for details.

Dashboard: Overview of all birds with stats: total distance, average speed, max altitude, stopover count, and tracking dates.

User Management: Admin panel to view all registered users, change their roles (admin or researcher), and remove accounts.

Data Upload: Upload new Excel GPS data files. The system preprocesses them automatically and updates the map and stats.

Timeline & Charts: Time-series charts of speed and altitude for each bird over the tracking period.

Authentication: Secure login with bcrypt-hashed passwords. Two roles: admin (full access) and researcher (view only).

8. How to Run the Application (Windows)

Clone from GitHub

```

git clone https://github.com/Nirmit2002/birdtrecking.git
cd birdtrecking

```

Install Python packages

```

pip install -r requirements.txt
pip install pymysql cryptography

```

Add MySQL to PATH (PowerShell)

```

$env:PATH += ";C:\Program Files\MySQL\MySQL Server 8.0\bin"

```

Set up the database

```
mysql -u root -p -e "CREATE DATABASE stork_db; CREATE USER 'stork_user'@'localhost' IDENTIFIED BY 'stork123';"
Get-Content stork_db.sql | mysql -u root -p stork_db
```

Run the app

```
python backend/app.py
```

Then open **http://localhost:5000** in any browser.

Role	Username	Password
Admin	admin	admin123
Researcher	fenil	fenil123